

CAHIER DES CHARGES

Plateforme de Blogging Avancée

Projet	Plateforme de Blogging Avancée
Type	Application Web Full-Stack
Durée	6 mois
Stack	Next.js · MySQL · Docker
Hébergement	DigitalOcean (GitHub Student Pack) · Domaine Namecheap .me
Version	1.1
Date	24/04/2026

1. Présentation du Projet

1.1 Contexte

Ce document constitue le cahier des charges d'un projet de stage d'une durée de 6 mois. L'objectif initial: une plateforme de blogging avec gestion des utilisateurs et base de données, a été volontairement enrichi afin de produire un projet à forte valeur technique, démontrable en entretien et présentable dans un portfolio professionnel.

1.2 Objectifs

- Concevoir et développer une plateforme de blogging complète et fonctionnelle
- Intégrer des fonctionnalités avancées au-delà du simple CRUD
- Déployer l'application sur un environnement cloud accessible publiquement
- Produire une base de code propre, documentée et containerisée
- Constituer un projet défendable et démontrable en entretien technique

1.3 Périmètre

Le projet couvre l'intégralité du cycle de développement : conception, implémentation, tests avec données simulées, déploiement sur DigitalOcean et livraison sous forme de dépôt GitHub public. L'application sera accessible via un domaine .me personnalisé avec HTTPS.

2. Stack Technique

2.1 Frontend & Backend

Next.js est utilisé comme framework full-stack unifié. Il prend en charge à la fois le rendu des pages React côté client et les routes API côté serveur, éliminant le besoin d'un serveur backend séparé. L'ensemble du code (pages, composants, et routes API) réside dans un seul projet.

- Next.js : framework React full-stack (pages + API routes)
- Tailwind CSS: utilitaires CSS pour le style, orienté esthétique brutaliste

2.2 Base de données

- MySQL: base de données relationnelle choisie pour la nature fortement structurée et relationnelle des données
- Indexation full-text native pour la fonctionnalité de recherche avancée
- Schéma normalisé : utilisateurs, posts, commentaires, likes, scores de sentiment, catégories

2.3 Infrastructure locale

- Docker: containerisation de l'application Next.js et de la base de données MySQL
- Docker Compose: orchestration des services en local et en production

- Docker Volumes: persistance des données MySQL et des fichiers uploadés entre redémarrages

2.4 Hébergement

- DigitalOcean Droplet: VPS Linux (Ubuntu) financé par les crédits du GitHub Student Developer Pack (\$200 / 1 an)
- Namecheap: domaine .me gratuit 1 an via le GitHub Student Developer Pack
- Nginx: reverse proxy exposé sur les ports 80 et 443, redirige les requêtes vers Next.js
- SSL / HTTPS: certificat Namecheap inclus ou Let's Encrypt

2.5 Service externe

- API NLP tierce (ex. Hugging Face Inference API, Google Natural Language API), seule dépendance externe, utilisée pour l'analyse de sentiment des commentaires

3. Modules Fonctionnels

Module	Fonctionnalités principales	Priorité
Authentification & Utilisateurs	Inscription, connexion, JWT + refresh tokens, deux rôles : Auteur / Lecteur, gestion du profil	Essentielle
Gestion du contenu	Création et édition de posts via éditeur rich text, upload d'image optionnel, catégories, états Brouillon / Publié	Essentielle
Commentaires	Soumission de commentaires, likes sur commentaires	Essentielle
Système de likes	Like / unlike sur posts et commentaires, un like par utilisateur par entité	Essentielle
Analyse de sentiment	Déclenchement à la soumission d'un commentaire, appel API NLP, score 0–1 stocké en base	Importante
Tableau de bord analytique	Tendance de sentiment par post et catégorie, croisement likes / sentiment, posts les plus controversés	Importante
Recherche avancée	Full-text search sur titres et contenus, filtres par catégorie et date, tri par date ou nombre de likes	Importante
Seed script	Génération de données fictives via Faker.js, spread intentionnel des sentiments, usage dev / démo uniquement	Secondaire

4. Exigences Fonctionnelles Détaillées

4.1 Authentification & Gestion des utilisateurs

- Inscription avec email et mot de passe (hashé via bcrypt, jamais stocké en clair)
- Connexion avec émission d'un JWT court et d'un refresh token longue durée
- Deux rôles distincts : Auteur (peut créer et gérer ses posts, accéder à son tableau de bord), Lecteur (peut commenter et liker)
- Middleware de contrôle d'accès appliqué à chaque route API protégée
- Mise à jour du profil utilisateur (nom, avatar, bio)

4.2 Gestion du contenu

- Éditeur rich text pour la rédaction des posts
- Upload d'image optionnel, fichier stocké dans un volume Docker, URL enregistrée en base
- Durée de lecture estimée affichée (calcul statique : nombre de mots / 200)

4.3 Commentaires & Engagement

- Soumission d'un commentaire par tout utilisateur authentifié
- La soumission déclenche immédiatement un appel à l'API NLP
- Le score de sentiment retourné (0.0 à 1.0) est stocké aux côtés du commentaire en base
- Likes sur posts et commentaires, contrainte d'unicité par (utilisateur, entité)

4.4 Tableau de bord analytique (Auteur)

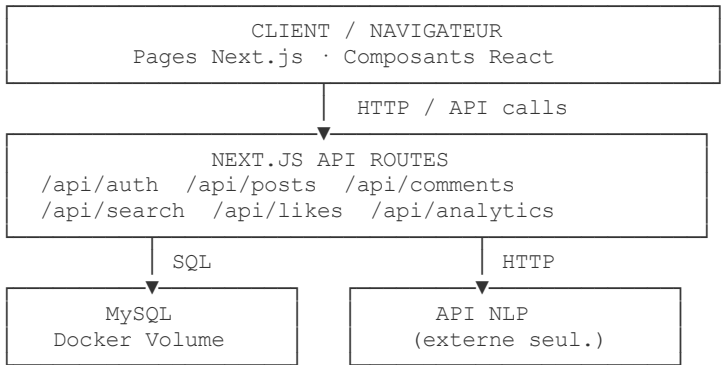
- Score de sentiment moyen par post sur une période glissante
- Graphique de tendance: évolution du sentiment dans le temps
- Vue croisée likes / sentiment: identifier les posts populaires mais ayant reçu des réactions mitigées, et inversement
- Posts les plus controversés: faible score de sentiment malgré un fort volume de commentaires

4.5 Recherche avancée

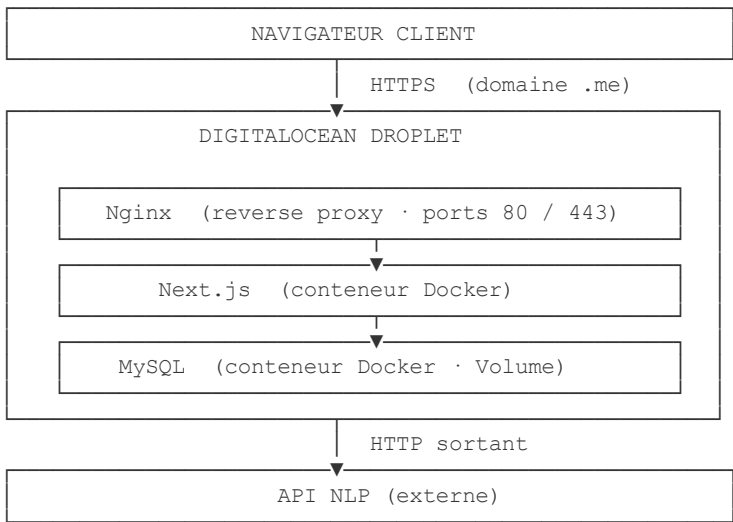
- Recherche full-text native MySQL sur titres et contenus des posts
- Filtre par période
- Tri par date de publication ou nombre de likes

5. Architecture Système

5.1 Architecture applicative



5.2 Architecture de déploiement



6. Hébergement & Déploiement

6.1 Infrastructure

Fournisseur cloud	DigitalOcean, \$200 de crédits via le GitHub Student Developer Pack
Serveur	Droplet Linux Ubuntu, VPS configuré manuellement
Domaine	Namecheap, domaine .me gratuit 1 an (GitHub Student Pack)

SSL / HTTPS	Certificat Namecheap inclus ou Let's Encrypt (renouvellement automatique)
Reverse proxy	Nginx, ports 80 et 443, redirige vers le port interne Next.js
Containerisation	Docker Compose, même fichier qu'en local, aucune configuration spécifique cloud
Persistence	Docker Volumes, données MySQL et fichiers uploadés survivent aux redémarrages
Coût estimé	Droplet \$6/mois, largement couvert par les \$200 de crédits sur la durée du projet

6.2 Étapes de déploiement

- Création du Droplet DigitalOcean (Ubuntu LTS) et configuration de l'accès SSH
- Installation de Docker et Docker Compose sur le serveur
- Clonage du dépôt GitHub sur le Droplet
- Configuration des variables d'environnement (.env) directement sur le serveur
- Lancement de l'application via docker-compose up -d
- Installation et configuration de Nginx comme reverse proxy
- Pointage du domaine Namecheap vers l'IP publique du Droplet (enregistrement DNS A)
- Activation HTTPS via le certificat SSL — redirection HTTP → HTTPS

7. Planning Prévisionnel

Phase	Durée	Contenu
Phase 1	Mois 1–2	Fondations : authentification complète (JWT + deux rôles), modèle de données, CRUD posts et commentaires, interface de base, Docker Compose local
Phase 2	Mois 2–3	Engagement : système de likes, catégories, upload d'images, éditeur rich text, recherche full-text basique
Phase 3	Mois 3–4	Recherche avancée : filtres par catégorie et date, tri par date et likes, indexation full-text MySQL
Phase 4	Mois 4–5	Sentiment & Analytics : intégration API NLP, stockage des scores, tableau de bord auteur (tendances, croisement likes/sentiment, posts controversés)
Phase 5	Mois 5–6	Finalisation & Déploiement : seed script, polish UI, DigitalOcean, Nginx, domaine, HTTPS, README, ADR

8. Livrables

- Application déployée et accessible via une URL publique (domaine .me, HTTPS)
- Dépôt GitHub public avec code source complet et historique de commits propre
- Fichier docker-compose.yml permettant un démarrage en une commande en local
- README détaillé: installation locale, variables d'environnement, description des modules, lien vers la version live
- Seed script documenté pour peupler la base avec des données réalistes (Faker.js)
- Documentation des décisions architecturales (ADR. Architecture Decision Records)

9. Contraintes & Hypothèses

- Hébergement financé intégralement par les crédits DigitalOcean du GitHub Student Developer Pack (\$200 / 1 an). Aucun coût prévu sur la durée du projet
- Domaine .me Namecheap gratuit 1 an via le GitHub Student Developer Pack
- Seule dépendance externe potentiellement payante: l'API NLP, des alternatives gratuites existent (Hugging Face Inference API)
- Projet individuel, aucune contrainte d'équipe ou de workflow collaboratif
- Base de données MySQL, choix définitif, justifié par la nature relationnelle des données
- Données de test entièrement simulées via Faker.js, aucun utilisateur réel requis

10. Glossaire

JWT	JSON Web Token — mécanisme d'authentification stateless
Refresh token	Token longue durée permettant de renouveler un JWT expiré sans reconnexion
NLP	Natural Language Processing — traitement automatique du langage naturel
Sentiment score	Score entre 0.0 et 1.0 indiquant la polarité d'un texte (0 = négatif, 1 = positif)
Seed script	Script générant des données fictives réalistes pour les tests et démonstrations
Docker Volume	Mécanisme Docker assurant la persistance des données entre redémarrages de conteneurs
Full-text search	Recherche dans le contenu intégral des documents, via l'indexation native MySQL
ADR	Architecture Decision Record — document retraçant une décision technique et sa justification
Droplet	Terme DigitalOcean désignant un serveur privé virtuel (VPS) Linux

Nginx	Serveur web utilisé ici comme reverse proxy devant Next.js
Reverse proxy	Intermédiaire réseau recevant les requêtes HTTPS et les redirigeant vers le bon service interne
DNS A record	Enregistrement DNS faisant pointer un nom de domaine vers une adresse IP publique
VPS	Virtual Private Server — serveur dédié virtualisé hébergé dans le cloud

— *Fin du document* —